Herencia y polimorfismo

Jerarquías de clases, enlace dinámico y colecciones polimórficas

Guía teórica de repaso para la tutoría

EIF201 Programación I · II Ciclo 2026 · Sesión combinada (semanas 9 y 15)
Andrey Jesús Solano Rojas, tutor

1. Propósito y alcance

Esta guía reúne en una sola sesión los dos temas que el cronograma del curso presenta por separado: la *herencia* (semana 9) y el *polimorfismo* con `dynamic_cast` y colecciones polimórficas (semana 15). Se tratan juntos porque el polimorfismo solo tiene sentido sobre una jerarquía de herencia, y porque ambos temas son la base del Proyecto 2 y del Examen 2.

Al finalizar el repaso, el estudiante debería ser capaz de:

- diseñar una jerarquía de clases aplicando generalización y especialización, y representarla en UML;
- invocar correctamente los constructores del padre y predecir el orden de constructores y destructores;
- distinguir sobrecarga, sobrescritura por sustitución y sobrescritura por extensión;
- explicar la diferencia entre enlace estático y enlace dinámico, y el papel de `virtual`;
- definir clases abstractas e interfaces mediante métodos virtuales puros;
- implementar una colección polimórfica que libere correctamente su memoria;
- usar `dynamic_cast` para acceder a lo específico de un tipo, y reconocer cuándo no hace falta.

Conocimientos previos. Clases, constructores y destructores, punteros, memoria dinámica, paso por referencia, miembros `static` y colecciones con arreglos dinámicos de punteros.

Ejemplo conductor. Toda la guía usa la misma jerarquía: una planilla de empleados de distintos tipos. Su diagrama de clases está en la figura 1 (sección 2). El código completo, verificado sin fugas de memoria, se entrega en `empleados.cpp`; los ejemplos cortos de enlace y de constructores están en `demos.cpp`.

2. Herencia

2.1 Generalización y especialización

La herencia permite definir una clase nueva (*derivada* o *hija*) a partir de otra existente (*base* o *padre*). Se llega a ella por dos caminos:

- **Generalización**: se observa que varias clases repiten atributos y comportamiento, y lo común se extrae a una clase padre. `EmpleadoFijo` y `EmpleadoPorHoras` comparten cédula, nombre y la forma de presentarse; eso sube a `Empleado`.
- **Especialización**: se parte de una clase existente y se crea una versión más específica. `Gerente` es un `EmpleadoFijo` que además recibe un bono.

La prueba para decidir si la herencia es adecuada es la frase **"es un"**: un gerente *es un* empleado fijo, y un empleado por horas *es un* empleado. Si la frase natural es "tiene un" (una planilla *tiene* empleados), la relación es de asociación, agregación o composición, no de herencia.

La figura 1 muestra la jerarquía completa del ejemplo conductor.

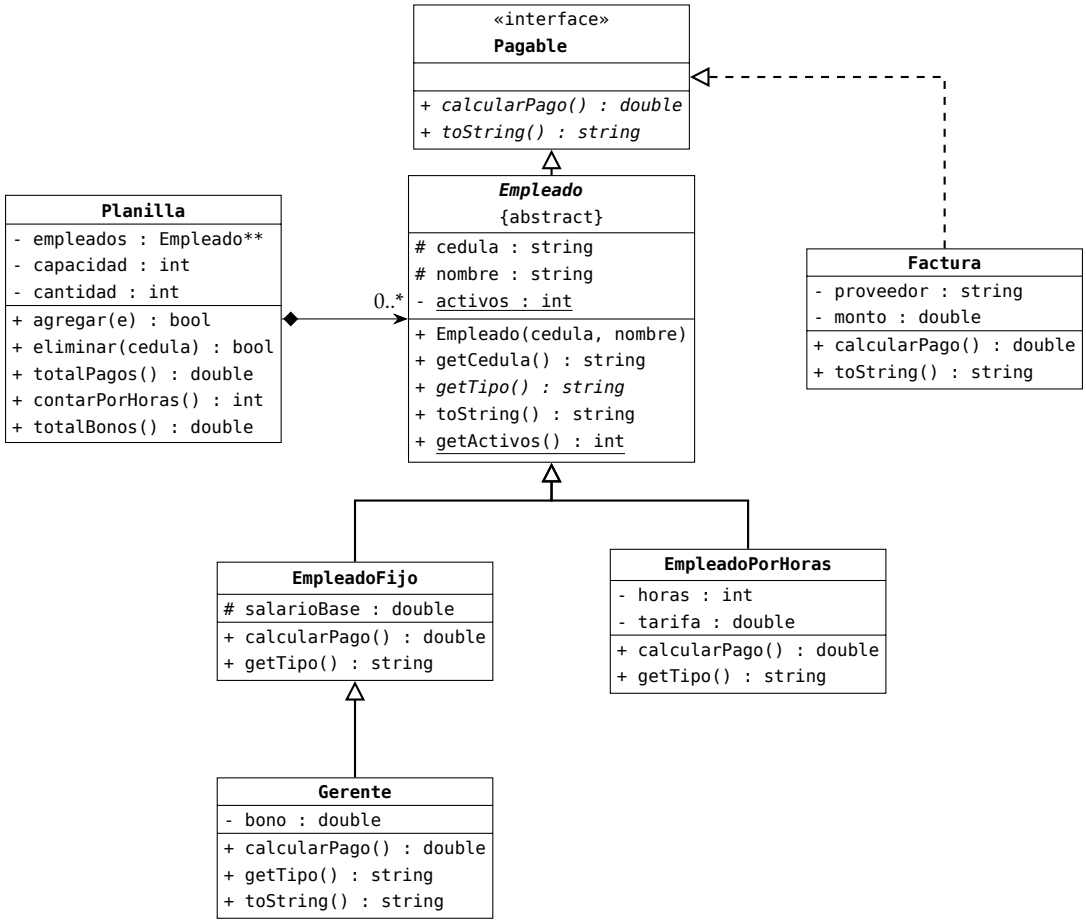


Figura 1. Diagrama de clases del ejemplo conductor. Los nombres en cursiva son abstractos; los subrayados son miembros de clase (**`static`**).

2.2 Sintaxis y reutilización

```
class EmpleadoFijo : public Empleado {
protected:
    double salarioBase;
public:
    EmpleadoFijo(string cedula, string nombre, double salarioBase)
        : Empleado(cedula, nombre), salarioBase(salarioBase) {}
    double getSalarioBase() const { return salarioBase; }
    void setSalarioBase(double salarioBase) { this->salarioBase = salarioBase; }
    double calcularPago() const override { return salarioBase; }
    string getTipo() const override { return "Fijo"; }
};
```

Con herencia pública, la clase derivada recibe todos los atributos y métodos de la base. Lo que cambia es quién puede *acceder* a cada miembro:

Miembro de la base	Dentro de la base	Dentro de la derivada	Desde fuera
`public`	Sí	Sí	Sí
`protected`	Sí	Sí	No
`private`	Sí	No (existe, pero no es accesible)	No

Por eso `Empleado` declara `cedula` y `nombre` como **`protected`**: las derivadas los pueden usar directamente sin exponerlos al resto del programa. Si fueran **`private`**, las derivadas tendrían que usar los *getters*, lo cual también es válido y a veces preferible.

2.3 Constructores y destructores

Los constructores **no se heredan**. Cada clase derivada inicializa lo propio y delega en el constructor del padre la inicialización de lo heredado, desde la lista de inicialización:

```
EmpleadoFijo(string cedula, string nombre, double salarioBase)
    : Empleado(cedula, nombre), salarioBase(salarioBase) {}
```

Si se omite la llamada, el compilador intenta usar el constructor sin parámetros del padre; como `Empleado` no lo tiene, el programa no compila. El orden de ejecución es siempre el mismo:

- **Constructores: de arriba hacia abajo.** Primero la base, después cada nivel derivado. Un objeto no puede construir su parte específica sin tener lista la parte heredada.
- **Destructores: en orden inverso.** Primero la derivada, al final la base.

```
1  class A {
2  public:
3      A() { cout << "A() "; }
4      virtual ~A() { cout << "~A() "; }
5  };
6
7  class B : public A {
8  public:
9      B() { cout << "B() "; }
10     ~B() { cout << "~B() "; }
11 };
12
13 class C : public B {
14 public:
15     C() { cout << "C() "; }
16     ~C() { cout << "~C() "; }
17 };
```

```
{ C c; }              // objeto automático: se destruye al cerrar la llave
A* p = new C();
delete p;             // funciona bien porque ~A() es virtual (sección 5)
```

Salida:

```
A() B() C() ~C() ~B() ~A()
A() B() C() ~C() ~B() ~A()
```

2.4 Sobrescritura: sustitución y extensión

Una clase derivada *sobrescribe* un método cuando define otro con la misma firma (mismo nombre, mismos parámetros y mismo calificador `const`). El programa del curso llama a esto *sobrescritura* (*overriding*) y distingue dos usos:

- **Sustitución**: la versión nueva reemplaza por completo a la del padre. `EmpleadoPorHoras::calcularPago()` no usa nada del padre.
- **Extensión**: la versión nueva *reutiliza* la del padre con el operador de ámbito `Padre::metodo()` y le agrega algo.

```
class Gerente : public EmpleadoFijo {
private:
    double bono;
public:
    Gerente(string cedula, string nombre, double salarioBase, double bono)
        : EmpleadoFijo(cedula, nombre, salarioBase), bono(bono) {}
    double getBono() const { return bono; }
    double calcularPago() const override {
        return EmpleadoFijo::calcularPago() + bono;      // extensión
    }
    string getTipo() const override { return "Gerente"; }
    string toString() const override {
        stringstream s;
        s << fixed << setprecision(0);
        s << EmpleadoFijo::toString() << " (incluye bono de " << bono << ")";
        return s.str();
    }
};
```

`Gerente::calcularPago()` extiende el cálculo del padre sumando el bono, y `Gerente::toString()` extiende la presentación. Observe que `EmpleadoFijo` no define `toString()`: la llamada `EmpleadoFijo::toString()` encuentra la versión heredada de `Empleado`.

> **Error clásico.** Olvidar el prefijo `EmpleadoFijo::` dentro de `Gerente::calcularPago()` convierte la llamada en `this->calcularPago()`, que vuelve a ejecutar `Gerente::calcularPago()`: una recursión infinita que termina con el programa caído.

3. Polimorfismo

Polimorfismo significa "muchas formas": una misma llamada, escrita una sola vez, produce comportamientos distintos según el objeto que la recibe. En C++ se logra combinando tres elementos: una jerarquía de herencia, punteros o referencias a la clase base y métodos `virtual`.

3.1 ¿Qué problema resuelve?

Sin polimorfismo, una planilla con empleados de varios tipos suele terminar así:

```
// Diseño SIN polimorfismo (lo que se quiere evitar)
double Planilla::totalPagos() const {
    double total = 0;
    for (int i = 0; i < cantidad; i++) {
        if (empleados[i]->getTipo() == "Fijo")
            total += empleados[i]->getSalarioBase();
        else if (empleados[i]->getTipo() == "Por horas")
            total += empleados[i]->getHoras() * empleados[i]->getTarifa();
        else if (empleados[i]->getTipo() == "Gerente")
            total += empleados[i]->getSalarioBase() + empleados[i]->getBono();
    }
    return total;
}
```

Cada tipo nuevo de empleado obliga a modificar este método y todos los que se parezcan a él. Con polimorfismo, la planilla le *pide* a cada empleado su pago y cada uno responde según su clase:

```
1  double Planilla::totalPagos() const {
2      double total = 0;
3      for (int i = 0; i < cantidad; i++)
4          total += empleados[i]->calcularPago();   // enlace dinámico
5      return total;
6  }
```

Este método no cambia aunque se agreguen veinte tipos de empleado. Es la idea del **principio abierto-cerrado** (tema V del curso): las clases deben estar *abiertas a la extensión* (se pueden agregar derivadas) y *cerradas a la modificación* (no hay que tocar el código que ya funciona).

3.2 Punteros y referencias a la clase base

Un puntero a la clase base puede apuntar a un objeto de cualquier clase derivada, porque todo `Gerente` *es un* `Empleado`. La conversión es implícita (*upcasting*):

```
Empleado* e = new Gerente("3-3333-3333", "Marta", 900000, 250000);
Pagable&  p = *e;      // también con referencias
```

A través de `e` solo se pueden invocar los métodos que `Empleado` declara. `e->getBono()` no compila, aunque el objeto sea un gerente: el compilador solo conoce el tipo del puntero.

3.3 `virtual` y el enlace dinámico

Cuando se invoca un método a través de un puntero o una referencia a la base, hay dos posibilidades:

- **Enlace estático** (método *no* `virtual`): el compilador decide qué versión ejecutar según el *tipo del puntero*.
- **Enlace dinámico** (método `virtual`): la decisión se toma en tiempo de ejecución según el *tipo real del objeto*.

```cpp
class Animal {
public:
    void presentarse() const { cout << "Soy un animal" << endl; }   // NO virtual
    virtual void sonido() const { cout << "..." << endl; }          // virtual
    virtual ~Animal() {}
};

class Perro : public Animal {
public:
    void presentarse() const { cout << "Soy un perro" << endl; }    // oculta
    void sonido() const override { cout << "Guau" << endl; }        // sobrescribe
};
```

```cpp
    Perro p;
    Animal* a = &p;
    Animal& r = p;
    Animal copia = p;          // se copia solo la parte Animal

    p.presentarse();           // 1
    a->presentarse();          // 2
    a->sonido();               // 3
    r.sonido();                // 4
    copia.sonido();            // 5
```

Línea	Llamada	Imprime	Motivo
1	`p.presentarse()`	Soy un perro	Se llama sobre el objeto mismo, de tipo `Perro`.
2	`a->presentarse()`	Soy un animal	No es `virtual`: manda el tipo del puntero (enlace estático).
3	`a->sonido()`	Guau	Es `virtual`: manda el tipo del objeto (enlace dinámico).
4	`r.sonido()`	Guau	Igual que con punteros, también funciona con referencias.
5	`copia.sonido()`	...	`copia` es un `Animal`: hubo rebanado (sección 3.4).

Basta con declarar `virtual` el método en la clase base; en las derivadas sigue siendo virtual aunque no se repita la palabra. La palabra `override` (C++11) es opcional, pero muy recomendable: le pide al compilador que verifique que de verdad se está sobrescribiendo un método virtual. Si la firma

no coincide (por ejemplo, falta el `const`), el compilador da un error en lugar de crear en silencio un método distinto.

¿Cómo funciona por dentro?

Los compiladores implementan el enlace dinámico con una *tabla de métodos virtuales* (*vtable*) por clase y un puntero oculto (*vptr*) en cada objeto. Llamar a un método virtual consiste en seguir el *vptr* y buscar en la tabla la dirección del método. Es un detalle de implementación que el estándar no exige, pero ayuda a explicar por qué la decisión depende del objeto y no del puntero.

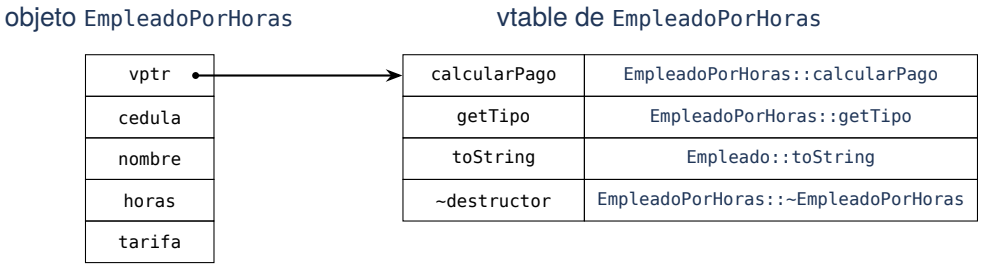


Figura 2. `toString` no se sobrescribe en `EmpleadoPorHoras`, así que su entrada apunta a la versión heredada.

3.4 Rebanado de objetos (*slicing*)

El polimorfismo funciona con **punteros y referencias**, nunca con objetos copiados por valor. En `Animal copia = p;` se copia solo la parte `Animal` del perro: lo específico se "rebana" y el objeto resultante es un `Animal` genuino. Lo mismo ocurre al pasar un objeto por valor a una función que recibe la clase base, o al guardar objetos en un arreglo de la clase base (`Animal zoo[10]`). Por eso las colecciones polimórficas guardan *punteros*.

3.5 Sobrecarga, sobrescritura y ocultamiento

```
class Calculadora {
public:
    int sumar(int a, int b) const { return a + b; }
    int sumar(int a, int b, int c) const { return a + b + c; }
    double sumar(double a, double b) const { return a + b; }
};
```

	Qué es	Dónde	Se decide
Sobrecarga	Mismo nombre, *distintos* parámetros (`sumar` con dos o tres enteros).	En la misma clase.	Al compilar, por los argumentos.
Sobrescritura	Misma firma que un método `virtual` de la base.	En una clase derivada.	Al ejecutar, por el tipo del objeto.
Ocultamiento	Misma firma que un método *no* virtual de la base (`presentarse`).	En una clase derivada.	Al compilar, por el tipo del puntero.

El ocultamiento casi nunca es lo que se busca; suele ser un `virtual` olvidado.

4. Clases abstractas e interfaces

4.1 Métodos virtuales puros y clases abstractas

Un **método virtual puro** se declara con `= 0` y no tiene implementación en esa clase: solo establece que todas las derivadas concretas deben proporcionarla. Una clase con al menos un método virtual puro es una **clase abstracta**:

- No se pueden crear objetos de ella: `Empleado e("1", "Ana");` y `new Empleado(...)` no compilan.
- Sí puede tener atributos, constructores, destructor y métodos implementados.
- Sí se pueden declarar punteros y referencias de su tipo; es justamente para eso que existe.
- Una derivada que no implemente todos los métodos puros también es abstracta.

```
// Clase abstracta: tiene datos y código común, pero no implementa
// calcularPago() ni getTipo().
class Empleado : public Pagable {
protected:
    string cedula;
    string nombre;
private:
    static int activos;                      // miembro de clase
public:
    Empleado(string cedula, string nombre) : cedula(cedula), nombre(nombre) {
        activos++;
    }
    virtual ~Empleado() {
        activos--;
    }
    static int getActivos() { return activos; }
    string getCedula() const { return cedula; }
    string getNombre() const { return nombre; }
    virtual string getTipo() const = 0;      // virtual puro
    string toString() const override {
        stringstream s;
        s << fixed << setprecision(0);
        s << "[" << getTipo() << "] " << nombre << " (" << cedula << "): "
          << calcularPago();                 // se resuelve en ejecución
        return s.str();
    }
};

int Empleado::activos = 0;                   // definición del static
```

`Empleado` no puede decir cuánto se le paga a "un empleado" en general, pero sí sabe cómo presentarlo. Observe que su `toString()` llama a `getTipo()` y a `calcularPago()`, que son virtuales puros: al ejecutarse sobre un `EmpleadoPorHoras`, el enlace dinámico invoca las versiones de esa clase. Así, el código común queda en la base y lo variable en las derivadas.

> **Excepción importante.** Dentro de un **constructor** o un **destructor** el enlace dinámico no llega a las derivadas: mientras se construye la parte `Empleado`, el objeto todavía no es un `Gerente`. Llamar a un método virtual puro desde el constructor de la base es un error.

4.2 Clases interface

C++ no tiene una palabra reservada para las interfaces. Por convención, una **clase interface** es una clase abstracta que solo tiene métodos virtuales puros y un destructor virtual, sin atributos. Define un *contrato*: "cualquier cosa que se pueda pagar".

```
// Clase interface: solo métodos virtuales puros y un destructor virtual.
class Pagable {
public:
    virtual double calcularPago() const = 0;
    virtual string toString() const = 0;
    virtual ~Pagable() {}
};
```

```
// Otra clase que cumple la interface, sin ser un Empleado.
class Factura : public Pagable {
private:
    string proveedor;
    double monto;
public:
    Factura(string proveedor, double monto) : proveedor(proveedor), monto(monto) {}
    double calcularPago() const override { return monto; }
    string toString() const override {
        stringstream s;
        s << fixed << setprecision(0) << "[Factura] " << proveedor << ": " << monto;
        return s.str();
    }
};
```

`Factura` no es un empleado, pero cumple el mismo contrato. Gracias a eso, un mismo arreglo de `Pagable*` puede reunir empleados y facturas:

```
Pagable* pagos[3] = {
    new Factura("Electricidad", 85000),
    new Factura("Internet", 32000),
    new EmpleadoFijo("5-5555-5555", "Pedro", 500000)
};
double totalMes = 0;
for (int i = 0; i < 3; i++)
    totalMes += pagos[i]->calcularPago();      // 617000
```

	Clase abstracta	**Clase interface**
Atributos	Puede tener.	No tiene.
Métodos implementados	Puede tener.	No tiene (solo virtuales puros).
Constructores	Puede tener.	No los necesita.
Propósito	Compartir código y datos entre clases parecidas.	Definir un contrato que cumplen clases que pueden no estar relacionadas.
Relación en UML	Generalización (línea continua).	Realización (línea punteada).

5. El destructor virtual

Cuando se libera un objeto derivado mediante un puntero a la base, el destructor que se ejecuta depende de si el destructor de la base es virtual:

```
class Recurso {                        // destructor NO virtual
public:
    ~Recurso() { cout << "~Recurso() "; }
};

class Archivo : public Recurso {
public:
    ~Archivo() { cout << "~Archivo() "; }
};
```

```
    Recurso* r = new Archivo();
    delete r;                  // comportamiento indefinido
    cout << endl;
```

Salida observada:

```
~Recurso()
```

Como `~Recurso()` no es virtual, `delete r` se resuelve por el tipo del puntero y `~Archivo()` nunca se ejecuta. Formalmente es *comportamiento indefinido*; en la práctica, todo lo que la derivada debía liberar queda como fuga de memoria. Con `virtual ~A()`, en cambio, `delete p` ejecutó `~C()`, `~B()` y `~A()` (sección 2.3).

> **Regla.** Si una clase tiene al menos un método virtual, o está pensada para usarse como base, su destructor debe ser `virtual`, aunque esté vacío. En el ejemplo, `~Pagable()` y `~Empleado()` lo son.

6. Colecciones polimórficas

6.1 Arreglo de punteros a la clase base

Una colección polimórfica guarda punteros a la clase base, cada uno apuntando a un objeto de una derivada distinta. Es la misma estructura de las colecciones con arreglos dinámicos que ya conoce el estudiante; lo nuevo es que los objetos no son todos del mismo tipo.

```
class Planilla {
private:
    Empleado** empleados;      // arreglo dinámico de punteros a la clase base
    int capacidad;
    int cantidad;
public:
    Planilla(int capacidad);
    ~Planilla();
    bool agregar(Empleado* e);
    bool eliminar(string cedula);
    double totalPagos() const;
    int contarPorHoras() const;
    double totalBonos() const;
    string toString() const;
};
```

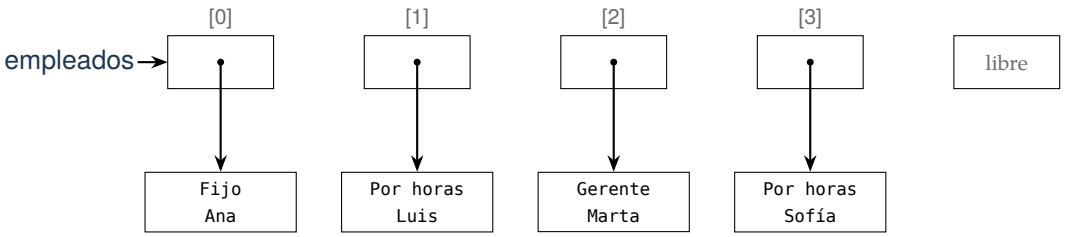


Figura 3. Arreglo dinámico de `Empleado*`: cada celda apunta a un objeto de una clase distinta.

```
Planilla::Planilla(int capacidad) : capacidad(capacidad), cantidad(0) {
    empleados = new Empleado*[capacidad];
}

Planilla::~Planilla() {
    for (int i = 0; i < cantidad; i++)
        delete empleados[i];         // destructor virtual: se ejecuta el correcto
    delete[] empleados;
}

bool Planilla::agregar(Empleado* e) {
    if (cantidad == capacidad)
        return false;
    empleados[cantidad++] = e;       // conversión implícita a la base
    return true;
}
```

Dos puntos clave:

- `agregar` recibe un `Empleado*`, así que acepta cualquier derivada sin cambiar nada.
- El destructor libera cada objeto con `delete empleados[i]`. Como `~Empleado()` es virtual, se ejecuta el destructor del tipo real de cada objeto. Después se libera el arreglo con `delete[]`.

No es posible declarar `Empleado empleados[10]`: `Empleado` es abstracta y, aunque no lo fuera, el arreglo de objetos rebanaría a cada empleado.

6.2 Eliminar de la colección

```
bool Planilla::eliminar(string cedula) {
    for (int i = 0; i < cantidad; i++) {
        if (empleados[i]->getCedula() == cedula) {
            delete empleados[i];
            for (int j = i; j < cantidad - 1; j++)
                empleados[j] = empleados[j + 1];
            cantidad--;
            return true;
        }
    }
    return false;
}
```

El orden importa: primero se libera el objeto y luego se corren los punteros siguientes. Si se corrieran primero, se perdería la dirección del objeto (fuga de memoria).

6.3 Acceder a lo específico: `dynamic_cast`

A veces se necesita algo que solo tiene un tipo derivado, como el bono de los gerentes. `dynamic_cast` convierte un puntero a la base en un puntero a la derivada *verificando en ejecución* que el objeto sea de ese tipo (o de uno derivado de él):

- Si el objeto es del tipo pedido, retorna un puntero válido.
- Si no lo es, retorna **`nullptr`**. Por eso el resultado siempre se compara antes de usarlo.
- Solo funciona con clases polimórficas (con al menos un método virtual).

```
double Planilla::totalBonos() const {
    double total = 0;
    for (int i = 0; i < cantidad; i++) {
        Gerente* g = dynamic_cast<Gerente*>(empleados[i]);
        if (g != nullptr)               // solo los gerentes tienen bono
            total += g->getBono();
    }
    return total;
}
```

```
int Planilla::contarPorHoras() const {
    int n = 0;
    for (int i = 0; i < cantidad; i++)
        if (dynamic_cast<EmpleadoPorHoras*>(empleados[i]) != nullptr)
            n++;
    return n;
}
```

> **Detalle que confunde.** `dynamic_cast<EmpleadoFijo*>` aplicado a un `Gerente` **no** retorna **`nullptr`**, porque un gerente *es un* empleado fijo. `dynamic_cast` responde "¿se puede tratar como...?", no "¿es exactamente...?".

`static_cast` no sirve para esto. También convierte de base a derivada, pero sin verificar nada: si el objeto no es del tipo indicado, el programa continúa con un puntero inválido.

Uso con moderación. Si un método está lleno de `dynamic_cast` para decidir qué hacer con cada tipo, se volvió al diseño con **`if`** de la sección 3.1. En ese caso suele faltar un método virtual en la base. El `dynamic_cast` es apropiado cuando la operación realmente solo tiene sentido para algunos tipos (como sumar bonos).

7. Miembros de instancia y de clase en una jerarquía

Un atributo `static` pertenece a la clase y existe una sola vez, compartido por todos los objetos de la clase *y de sus derivadas*. En el ejemplo, `Empleado::activos` cuenta los empleados vivos: el constructor de `Empleado` lo incrementa (se ejecuta al crear cualquier tipo de empleado) y el destructor lo decrementa (se ejecuta al liberar cualquiera de ellos, gracias al destructor virtual).

- Se declara dentro de la clase y se define una vez fuera: `int Empleado::activos = 0;`.
- Un método `static` se invoca sin objeto (`Empleado::getActivos()`) y no tiene `this`.
- Un método `static` **no puede ser virtual**: el enlace dinámico necesita un objeto, y el método estático no lo tiene.

8. Errores frecuentes

Error	Consecuencia	Corrección
No llamar al constructor del padre en la lista de inicialización	No compila si el padre no tiene constructor sin parámetros	`: Padre(args), atributo(valor).`
Olvidar `virtual` en la base	Se ejecuta la versión del padre (enlace estático)	Declarar `virtual` en la base y usar `override` en las derivadas.
Firma distinta en la derivada (por ejemplo, sin `const`)	No sobrescribe: crea otro método; si el de la base es puro, la derivada queda abstracta	Usar `override` para que el compilador lo detecte.
Destructor de la base no virtual	No se ejecuta el destructor de la derivada: fugas	`virtual ~Base() {}`.
Llamar a `metodo()` en lugar de `Padre::metodo()` al extender	Recursión infinita	Usar el operador de ámbito.
Pasar o guardar objetos por valor (`Animal a = perro`)	Rebanado: se pierde lo específico	Usar punteros o referencias.
Usar el resultado de `dynamic_cast` sin verificarlo	Desreferenciar `nullptr`	Comparar con `nullptr` antes de usarlo.
Usar `static_cast` para bajar en la jerarquía sin estar seguro del tipo	Puntero inválido, fallos difíciles de rastrear	Usar `dynamic_cast`.
Llamar a un método de la derivada con un puntero a la base (`e->getBono()`)	No compila	Agregar un método virtual a la base o usar `dynamic_cast`.
Correr los punteros antes de liberar al eliminar de la colección	Se pierde la dirección del objeto: fuga	Liberar primero, correr después.

9. Resumen

Concepto	Sintaxis	Para qué
Herencia	`class Hija : public Padre`	Reutilizar y especializar (relación "es un").
Constructor del padre	`Hija(...) : Padre(...), x(...) {}`	Inicializar la parte heredada.
Extensión	`Padre::metodo()` dentro de la versión nueva	Reutilizar la versión del padre.
Método virtual	`virtual double f() const;`	Enlace dinámico.
Sobrescritura verificada	`double f() const override;`	Detectar firmas incorrectas.
Virtual puro	`virtual double f() const = 0;`	Obligar a las derivadas a implementarlo.
Destructor virtual	`virtual ~Base() {}`	Liberar correctamente por puntero a la base.
Colección polimórfica	`Base** v = new Base*[n];`	Tratar objetos distintos de forma uniforme.
Conversión verificada	`dynamic_cast<Hija*>(p)`	Acceder a lo específico de un tipo.

Referencias

Deitel, H. y Deitel, P. (2016). *Cómo programar en C++* (11.ª ed.). Prentice Hall.

Joyanes, L. (1996). *Programación orientada a objetos: conceptos, modelado, diseño y codificación en C++*. McGraw-Hill.

A. Programa de prueba

El código completo del ejemplo conductor está en `empleados.cpp`. Compila con `g++ -std=c++11` o superior y fue verificado sin fugas de memoria. Su función `main`:

```
int main() {
    cout << fixed << setprecision(0);
    {
        Planilla planilla(10);
        planilla.agregar(new EmpleadoFijo("1-1111-1111", "Ana", 650000));
        planilla.agregar(new EmpleadoPorHoras("2-2222-2222", "Luis", 120, 3500));
        planilla.agregar(new Gerente("3-3333-3333", "Marta", 900000, 250000));
        planilla.agregar(new EmpleadoPorHoras("4-4444-4444", "Sofia", 80, 4000));
        cout << planilla.toString();
        cout << "Total de pagos:   " << planilla.totalPagos() << endl;
        cout << "Por horas:        " << planilla.contarPorHoras() << endl;
        cout << "Total de bonos:   " << planilla.totalBonos() << endl;
        cout << "Empleados activos: " << Empleado::getActivos() << endl;

        planilla.eliminar("2-2222-2222");
```

```
        cout << "Tras eliminar a Luis, activos: " << Empleado::getActivos() << endl;

        // Pagos del mes: la interface permite mezclar empleados y facturas
        Pagable* pagos[3] = {
            new Factura("Electricidad", 85000),
            new Factura("Internet", 32000),
            new EmpleadoFijo("5-5555-5555", "Pedro", 500000)
        };
        double totalMes = 0;
        for (int i = 0; i < 3; i++) {
            cout << pagos[i]->toString() << endl;
            totalMes += pagos[i]->calcularPago();
        }
        cout << "Total pagos del mes: " << totalMes << endl;
        for (int i = 0; i < 3; i++)
            delete pagos[i];            // ~Pagable es virtual
    }   // aquí se destruye la planilla y libera a sus empleados
    cout << "Al final, activos: " << Empleado::getActivos() << endl;
    return 0;
}
```

Salida del programa:

```
[Fijo] Ana (1-1111-1111): 650000
[Por horas] Luis (2-2222-2222): 420000
[Gerente] Marta (3-3333-3333): 1150000 (incluye bono de 250000)
[Por horas] Sofia (4-4444-4444): 320000
Total de pagos:   2540000
Por horas:        2
Total de bonos:   250000
Empleados activos: 4
Tras eliminar a Luis, activos: 3
[Factura] Electricidad: 85000
[Factura] Internet: 32000
[Fijo] Pedro (5-5555-5555): 500000
Total pagos del mes: 617000
Al final, activos: 0
```